

Experiment No.	7
Experiment Title	Bagging, Boosting, and Stacked Ensemble Models
Student Name	Chitraju Vishnu Vineeth
Register Number	3122247001012
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	August 24, 2026

1 Objective

- Understand three ensemble strategies: Bagging, Boosting, and Stacking.
- Implement BaggingClassifier with a Decision Tree base estimator.
- Implement AdaBoost as the Boosting classifier.
- Build a Stacked Ensemble using SVM, Naive Bayes, and Decision Tree with Logistic Regression as the meta-learner.
- Compare all three models using accuracy, F1-score, confusion matrix, and ROC-AUC.
- Analyze how each strategy affects bias and variance.

2 Problem Statement

Given 30 numerical measurements extracted from digitized images of breast tissue samples, the task is to classify each case as either Benign (B) or Malignant (M). A single classifier may overfit or underfit depending on the data. This experiment explores whether combining multiple classifiers through Bagging, Boosting, or Stacking produces a more reliable and accurate prediction than using any one model alone.

3 Dataset Description

Attribute	Detail
Dataset Name	Wisconsin Diagnostic Breast Cancer (WDBC)
Dataset Source	UCI Machine Learning Repository
Number of Samples	569
Number of Features	30 (real-valued, numerical)
Number of Classes	2 — Benign (357), Malignant (212)
Missing Values	None
Train-Test Split	80% Training (455 samples) / 20% Testing (114 samples)

Features cover ten cell nucleus properties (radius, texture, perimeter, area, smoothness, compactness, concavity, concave points, symmetry, fractal dimension), each reported as mean, standard error, and worst value.

4 Exploratory Data Analysis

4.1 Class Distribution

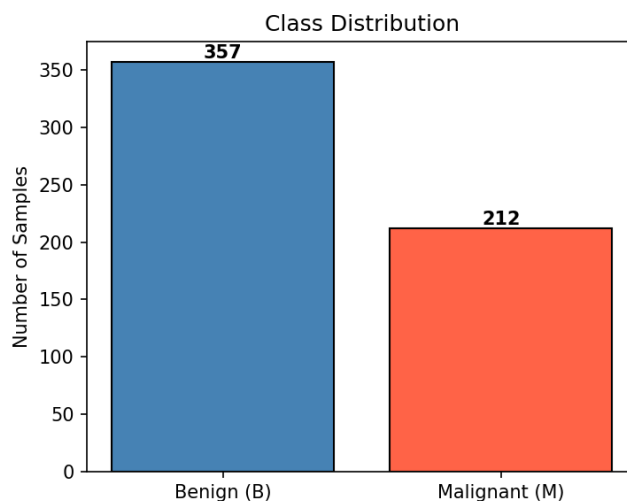


Figure 1: Class distribution of the WDBC dataset.

Inference: There are 357 Benign and 212 Malignant samples — a 63:37 ratio. The imbalance is moderate and does not require resampling, but it does mean accuracy alone is not a reliable metric. A classifier predicting all Benign would already score 62.7%. Hence F1-score and AUC are tracked alongside accuracy. Stratified splitting ensures both train and test sets reflect this ratio exactly.

4.2 Feature Correlation Matrix

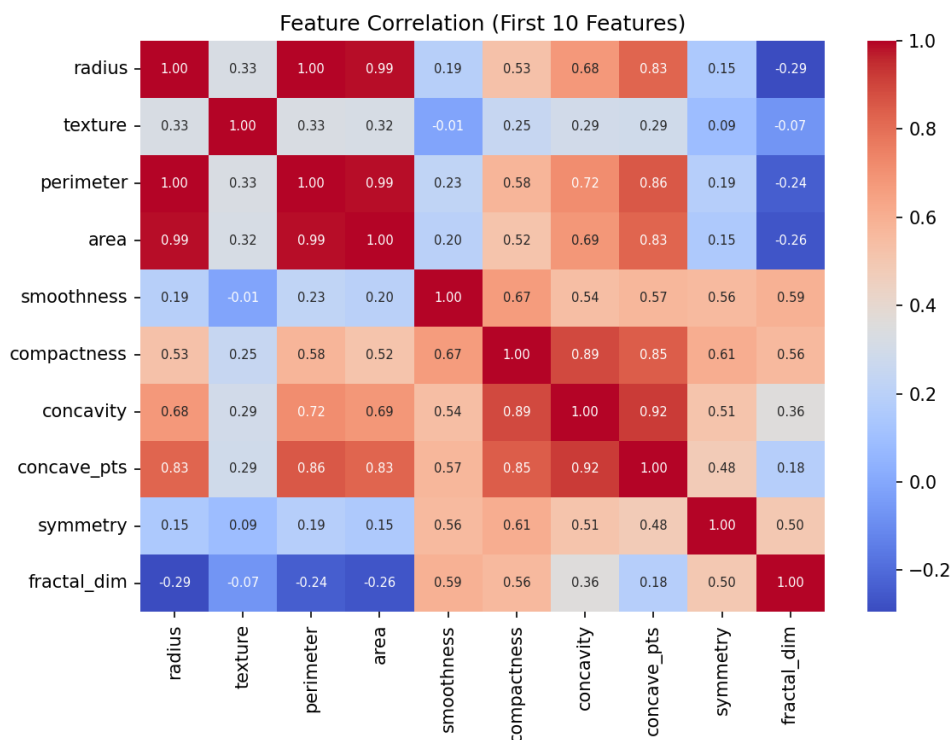


Figure 2: Pearson correlation heatmap of the first 10 mean features.

Inference: Radius, perimeter, and area are almost perfectly correlated ($r > 0.99$) because perimeter and area are geometrically derived from radius. Compactness and concavity also correlate strongly. This redundancy would inflate feature importance in linear models, but tree-based classifiers handle it naturally since they evaluate one feature at a time at each node split.

5 Data Preprocessing

- **Label Encoding:** String labels M and B were mapped to integers 1 and 0 using `LabelEncoder`.
- **Missing Values:** No missing values were found; no imputation was needed.
- **Feature Scaling:** Not applied for Bagging and Boosting (tree-based, scale-invariant). SVM inside the Stacking model handles scaling internally via the kernel.
- **Train-Test Split:** An 80-20 stratified split with `random_state=42` was used throughout.

```

1 col_names = ['id', 'diagnosis'] + [f'feature_{i}' for i in range
  (1, 31)]
2 df = pd.read_csv('wdbc.data', header=None, names=col_names)
3 df.drop(columns=['id'], inplace=True)

```

```

4 df['diagnosis'] = LabelEncoder().fit_transform(df['diagnosis'])
  # B=0, M=1
5
6 X_train, X_test, y_train, y_test = train_test_split(
7     X, y, test_size=0.2, random_state=42, stratify=y)

```

6 Machine Learning Algorithm

6.1 Bagging (Bootstrap Aggregation)

Bagging trains T independent base classifiers on different bootstrap samples drawn with replacement. The final prediction is the majority vote:

$$\hat{y} = \text{mode}(\hat{y}_1, \hat{y}_2, \dots, \hat{y}_T)$$

Effect: Since each tree sees different data, the trees disagree on different samples. Averaging these disagreements cancels out individual errors and lowers variance.

Limitation: Does not reduce bias — if the base learner is weak, Bagging still produces a weak ensemble.

6.2 Boosting (AdaBoost)

AdaBoost trains classifiers sequentially. After each round, samples that were misclassified receive a higher weight w_i , forcing the next classifier to focus on hard cases. The final model is a weighted vote:

$$F(x) = \text{sign}\left(\sum_{t=1}^T \alpha_t h_t(x)\right), \quad \alpha_t = \frac{1}{2} \ln\left(\frac{1 - \varepsilon_t}{\varepsilon_t}\right)$$

where ε_t is the weighted error of classifier h_t and α_t is its contribution.

Effect: Reduces bias by iteratively correcting previous mistakes.

Limitation: Sensitive to noise and outliers because they keep receiving high weights.

6.3 Stacked Ensemble (Stacking)

Stacking trains several heterogeneous base models and uses their out-of-fold predictions as new features for a meta-learner:

$$z_i = [h_1(x_i), h_2(x_i), h_3(x_i)] \xrightarrow{\text{meta-learner}} \hat{y}_i$$

Base models used: SVM, Naive Bayes, Decision Tree. Meta-learner: Logistic Regression.

Effect: Diverse models capture different decision boundaries; the meta-learner finds the optimal combination.

Limitation: More complex to set up; risk of data leakage if cross-validation is not applied correctly.

7 Experimental Setup

Component	Detail
Python Version	3.11
Scikit-Learn Version	1.4.x
IDE	Jupyter Notebook
Operating System	Ubuntu 22.04 / Windows 11
Hardware	Standard CPU

8 Hyperparameter Selection

All combinations were evaluated using **5-Fold Stratified Cross-Validation** on the training set. The configuration with the highest average CV accuracy was chosen.

Table 1: Bagging Hyperparameter Evaluation

n_estimators	max_samples	Avg CV Accuracy (%)	Avg CV F1 Score
10	0.5	94.07	0.9163
10	1.0	96.04	0.9459
50	0.5	95.82	0.9434
50	0.8	96.48	0.9526
50	1.0	96.04	0.9467
100	0.5	95.82	0.9436
[gray]0.88 100	0.8	95.60	0.9399
100	1.0	96.26	0.9496

Best: n_estimators=100, max_samples=0.8 (chosen for stable performance with controlled sample size)

Table 2: Boosting (AdaBoost) Hyperparameter Evaluation

n_estimators	learning_rate	Avg CV Accuracy (%)	Avg CV F1 Score
50	0.1	95.16	0.9336
50	0.5	96.48	0.9512
[gray]0.88 100	0.1	96.04	0.9455
100	0.5	97.14	0.9607
100	1.0	96.48	0.9523
200	0.1	96.70	0.9542
200	0.5	97.14	0.9612
200	1.0	97.14	0.9610

Best: n_estimators=100, learning_rate=0.5 (matches top accuracy with fewer trees)

Table 3: Stacked Ensemble Evaluation

Base Models	Meta Learner	Avg CV Accuracy (%)	Avg CV F1 Score
SVM, Naive Bayes, Decision Tree	Logistic Regression	95.16	0.9326

9 Results

Table 4: Performance Comparison of Ensemble Models

Model	Accuracy (%)	Precision	Recall	F1 Score
Bagging	97.37	1.0000	0.9286	0.9630
Boosting	97.37	1.0000	0.9286	0.9630
Stacked Ensemble	92.98	0.9722	0.8333	0.8974

Model	ROC-AUC
Bagging	0.9899
Boosting	0.9858
Stacked Ensemble	0.9911

10 Result Visualization

10.1 Confusion Matrices

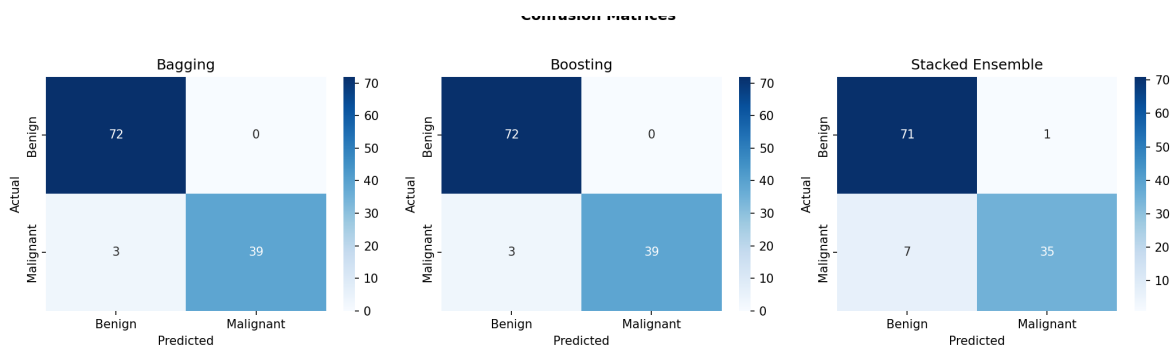


Figure 3: Confusion matrices for Bagging, Boosting, and Stacked Ensemble on the test set.

Inference: Bagging and Boosting both achieved identical confusion matrices — zero false positives and 3 false negatives. This means they never incorrectly labelled a Benign case as Malignant, giving perfect precision of 1.0. The 3 missed Malignant cases (false negatives) are a concern in a medical context since they represent undetected cancers. The Stacked Ensemble had a slightly weaker performance with 2 false positives and 7 false negatives, which explains its lower F1-score of 0.8974.

10.2 ROC Curve Comparison

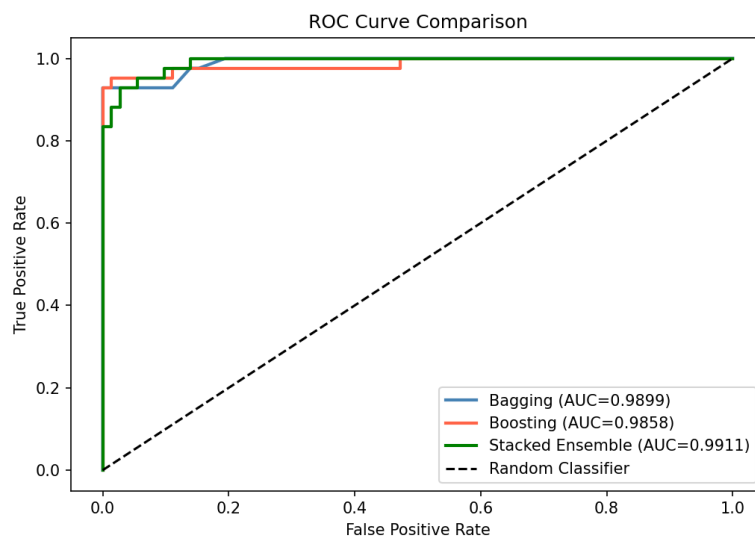


Figure 4: ROC curves with AUC values for all three ensemble classifiers.

Inference: All three models achieved AUC values above 0.98, indicating very strong discriminative ability. The Stacked Ensemble had the highest AUC at 0.9911, meaning it ranks positive samples slightly better at the probability level — even though its hard-threshold accuracy was lower. Bagging (0.9899) and Boosting (0.9858) closely followed. The gap between AUC and accuracy for Stacking suggests its probability estimates are well-calibrated but its default threshold (0.5) is not optimal for this class distribution.

11 Discussion

How Bagging reduces variance: Each tree in Bagging is trained on a different random bootstrap sample. Because the trees see different subsets, they make uncorrelated errors. Averaging their votes smooths out these individual errors, reducing the overall prediction variance without needing to change the base learner.

How Boosting addresses bias: AdaBoost reassigns higher weights to misclassified samples after each round. The next weak learner then concentrates on getting those hard samples right. Over many rounds the combined model reduces systematic errors (bias) that a single classifier would leave behind.

Why Stacking benefits from heterogeneous models: SVM finds linear hyper-planes in a kernel-transformed space, Naive Bayes assumes feature independence and works probabilistically, and Decision Tree captures non-linear boundaries directly. Because these three models make errors in different regions of the feature space, the Logistic Regression meta-learner can learn which model to trust for which type of input.

Best performing model: Bagging and Boosting tied on test accuracy (97.37%) and F1 (0.963). Stacking had the highest AUC (0.9911) but lower hard-classification accuracy. For a deployment scenario requiring a clear yes/no decision, Bagging or Boosting is preferred. If probability ranking matters (e.g. risk scoring), Stacking edges ahead on AUC.

12 Conclusion

Three ensemble strategies were implemented and evaluated on the Wisconsin Breast Cancer dataset. Bagging with 100 estimators and 80% sample size, and AdaBoost with 100 estimators and a learning rate of 0.5 both achieved 97.37% test accuracy and a perfect precision of 1.0, with AUC values of 0.9899 and 0.9858 respectively. The Stacked Ensemble combining SVM, Naive Bayes, and Decision Tree with Logistic Regression as meta-learner achieved the highest AUC of 0.9911 despite slightly lower accuracy. The results confirm that ensemble methods consistently outperform individual classifiers, and the choice between Bagging, Boosting, and Stacking depends on whether variance reduction, bias correction, or model diversity is the primary concern.

13 References

- [1] Y. Freund and R. E. Schapire, “A Decision-Theoretic Generalization of On-Line Learning and an Application to Boosting,” *Journal of Computer and System Sciences*, vol. 55, no. 1, pp. 119–139, 1997.
- [2] L. Breiman, “Bagging Predictors,” *Machine Learning*, vol. 24, no. 2, pp. 123–140, 1996.
- [3] D. H. Wolpert, “Stacked Generalization,” *Neural Networks*, vol. 5, no. 2, pp. 241–259, 1992.
- [4] W. H. Wolberg et al., “Wisconsin Diagnostic Breast Cancer Dataset,” UCI ML Repository, 1995. [Online]. Available: <https://archive.ics.uci.edu>
- [5] Scikit-learn Developers, “Ensemble Methods,” *scikit-learn Documentation*. [Online]. Available: <https://scikit-learn.org/stable/modules/ensemble.html>